Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. Organización del espacio de direcciones virtuales en GNU/Linux
2. Análisis del espacio de direcciones de una tarea
3. Asignación dinámica de memoria
3.1. El servicio mmap de POSIX
3.2. El servicio munmap de POSIX
3.3. Las funciones malloc() y free()
4. La tabla de páginas en GNU/Linux
Archivos de la práctica
Ejercicios

Organización de la memoria de las tareas en GNU/Linux

Área de Arquitectura y Tecnología de Computadores – Versión 1.2.12026, 23/10/2023

Objetivos de la sesión

Esta sesión práctica tiene los siguientes objetivos:

- Mostrar la organización del espacio de direcciones virtuales de las tareas.
- Analizar la independencia de los espacios de direcciones virtuales de las tareas.
- Analizar los cambios que se producen en los espacios de direcciones de las tareas durante la petición y liberación de memoria en Linux.
- Analizar las propiedades de las diferentes áreas en las que se organiza la memoria de las tareas y el sistema operativo.

Conocimientos y materiales necesarios

Para poder realizar esta sesión:

- El alumno debe conocer los principios básicos de funcionamiento de la memoria virtual.
- Es necesario disponer de una versión de GNU/Linux con el módulo *linmem* [1] instalado, necesario en esta sesión para obtener la entrada en tabla de páginas (ETP) asociada a cualquier dirección virtual de una tarea. La máquina virtual suministrada en la asignatura ya dispone de este módulo instalado.
- Durante la sesión se plantearán una serie de preguntas que puedes responder en el correspondiente cuestionario en el campus virtual. Puedes abrir el cuestionario en otra pestaña del navegador pinchando en el enlace mientras mantienes pulsada la tecla `Ctrl`.

1. Organización del espacio de direcciones virtuales en GNU/Linux

- Arranca Linux.
- Cambia al directorio de tu repositorio y sincronízalo con el repositorio de Bitbucket para actualizar los cambios que se hubiesen realizado con anterioridad.

```
$> git pull
```

- Descarga los ficheros necesarios para la práctica y descomprímelos con estas órdenes:

```
$> wget http://rigel.atc.uniovi.es/grado/2ac/files/sesion3-4.tar.gz
$> tar xvfz sesion3-4.tar.gz
```

- Añade los ficheros al índice git, confirma los cambios y súbelos al servidor con estas órdenes:

```
$> git add sesion3-4
$> git commit
$> git push
```

- Cámbiate al directorio `sesion3-4`.

La máquina virtual de prácticas ejecuta una versión de Linux de 32 bits. En esta versión se asocia a cada tarea un espacio de direcciones virtuales de 2^{32} = 4 GiB. El GiB de memoria virtual más alto está reservado para el sistema operativo, mientras que los 3 GiB de memoria virtual más bajos están asociados a la tarea. Esta situación se muestra en la figura 1.

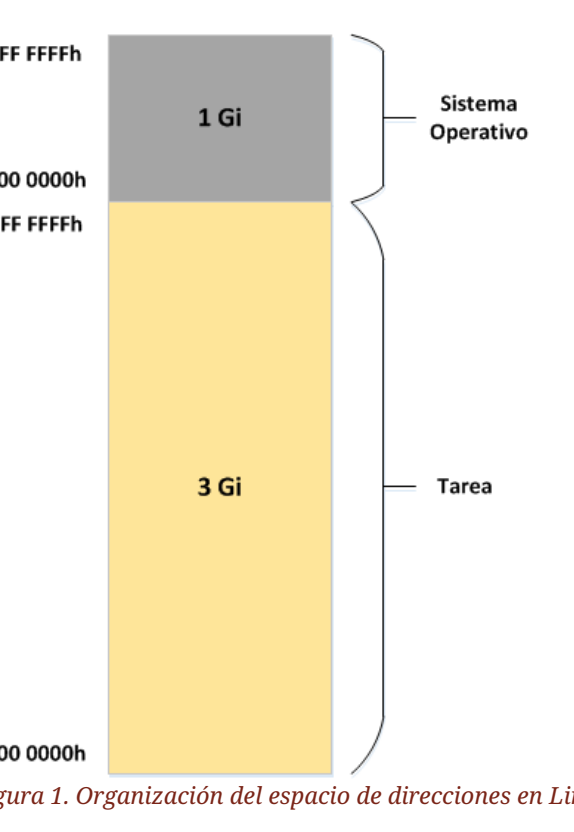


Figura 1. Organización del espacio de direcciones en Linux 32 bits

Obviamente, los 3 GiB de memoria asociados a la tarea no tienen asociada memoria en su totalidad y por lo tanto no son accesibles. Únicamente parte del espacio de direcciones virtuales de una tarea tiene asociada memoria. No se debería acceder a las direcciones virtuales que no tengan asociada memoria, pues cualquier intento de acceso generaría un fallo de página no recuperable del tipo `Violación de segmento` (*Segmentation fault*).

> A lo largo de esta sesión empleamos el término memoria como sinónimo de almacenamiento físico, aunque estrictamente no son la misma cosa. Debe tenerse en cuenta que el almacenamiento físico asociado a una tarea se divide entre la memoria y el archivo de paginación (o área de *swapping*).

Además, hay direcciones asociadas a la tarea que tienen usos especiales. Por ejemplo, la constante `NULL` en C se emplea para indicar punteros (direcciones de memoria) no inicializados y se implementa en la biblioteca estándar de C asignándole el valor `0x00000000`. Como el gestor de memoria de Linux nunca asigna memoria a la página virtual que contenga dicha dirección, cualquier intento de lectura o escritura en la posición de memoria referenciada con un puntero `NULL` genera un fallo de página no recuperable que llevaría al programador directamente a la instrucción causante. Esto es muy útil para la depuración de programas, pues un error de programación que implique por ejemplo la escritura usando un puntero no inicializado, podría provocar la escritura descontrolada en una variable del programa y generar grandes quebraderos de cabeza al programador.

Una vez hemos dejado claro que la tarea solo puede acceder a direcciones virtuales dentro del rango `00000000h–BFFFFFFFh`, resulta interesante establecer cómo se organiza la memoria dentro de dicho rango.

La organización del espacio de direcciones virtuales accesible a una tarea se encuentra en el archivo `/proc/<pid>/maps`, donde `<pid>` es un patrón de sustitución que hace referencia al identificador de la tarea (proceso). Para mostrarlo, lleva a cabo las siguientes operaciones:

- Compila y enlaza el archivo `3-4maps.c` proporcionado por el profesor, usando la orden

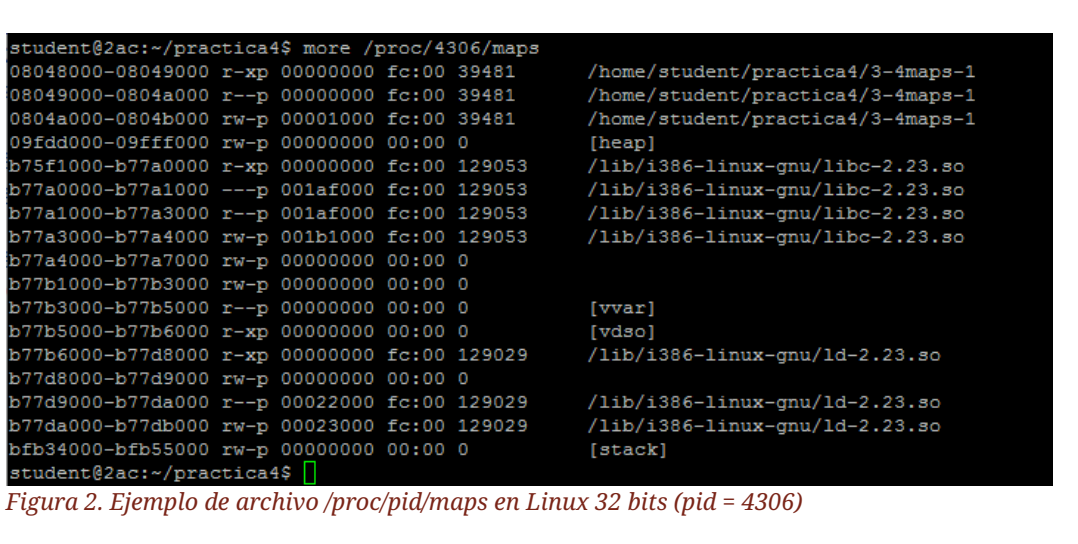
```
$> gcc 3-4maps.c -o 3-4maps-1
```

  Se trata simplemente de un programa que muestra por pantalla su `pid` y espera por la pulsación de una tecla para terminar, lo que nos permitirá observar su archivo *maps* antes de que termine.
- Ejecuta el programa `3-4maps-1`. No pulses la tecla `ENTER`.
- Desde una segunda terminal, ejecuta la siguiente orden

```
$> more /proc/<pid>/maps
```

  sustituyendo `<pid>` por el identificador de proceso mostrado por el programa. Una vez lo has hecho, ya puedes terminar la tarea pulsando la tecla `ENTER` en la primera interfaz de comandos.

En la terminal tendrás un resultado similar a la imagen mostrada en la figura 2.



```
student@2ac:~/practica4$ more /proc/4306/maps
08048000-08049000 r-xp 00000000 fc:00 39481      /home/student/practica4/3-4maps-1
08049000-0804a000 r--p 00000000 fc:00 39481      /home/student/practica4/3-4maps-1
0804a000-0804b000 rw-p 00001000 fc:00 39481      /home/student/practica4/3-4maps-1
09fdd000-09fff000 rw-p 00000000 00:00 0          [heap]
b75f1000-b77a0000 r-xp 00000000 fc:00 129053     /lib/i386-linux-gnu/libc-2.23.so
b77a0000-b77a1000 ---p 001af000 fc:00 129053     /lib/i386-linux-gnu/libc-2.23.so
b77a1000-b77a3000 r--p 001af000 fc:00 129053     /lib/i386-linux-gnu/libc-2.23.so
b77a3000-b77a4000 rw-p 001b1000 fc:00 129053     /lib/i386-linux-gnu/libc-2.23.so
b77a4000-b77a7000 rw-p 00000000 00:00 0
b77b1000-b77b3000 rw-p 00000000 00:00 0
b77b3000-b77b5000 r--p 00000000 00:00 0          [vvar]
b77b5000-b77b6000 r-xp 00000000 00:00 0          [vdso]
b77b6000-b77d8000 r-xp 00000000 fc:00 129029     /lib/i386-linux-gnu/ld-2.23.so
b77d8000-b77d9000 rw-p 00000000 00:00 0
b77d9000-b77da000 r--p 00022000 fc:00 129029     /lib/i386-linux-gnu/ld-2.23.so
b77da000-b77db000 rw-p 00023000 fc:00 129029     /lib/i386-linux-gnu/ld-2.23.so
bfb34000-bfb55000 rw-p 00000000 00:00 0          [stack]
student@2ac:~/practica4$
```

Figura 2. Ejemplo de archivo /proc/pid/maps en Linux 32 bits (pid = 4306)

Como podrás observar, en la primera columna del archivo `/proc/pid/maps` aparecen los diferentes rangos de memoria separados por un guion. La primera dirección del rango es la dirección virtual de comienzo, mientras que la segunda es la dirección final (más uno). Comprueba cómo todas estas direcciones se encuentran en la zona de memoria reservada para la tarea, esto es, el rango `00000000h–BFFFFFFFh`.

Observa además cómo el tamaño de cada una de las áreas mostradas es múltiplo de 4 KiB, el tamaño de la página. Esto es así pues la memoria se asigna en unidades de página. En la segunda columna aparecen los permisos de lectura (*r*), escritura (*w*) y ejecución (*x*) asociados al área de memoria. También se indica si el área de memoria es privada (*p*) o compartida por varias tareas (*s*). Finalmente, en la columna más a la derecha aparece el nombre del archivo asociado a cada área de memoria si hay alguno asociado, o bien un espacio en blanco para áreas anónimas gestionadas por el sistema. Hay zonas en las que no tiene sentido un nombre de archivo y en ese caso indican el tipo área, como por ejemplo `[vvar]`, `[heap]`, etc.

Estas son algunas de las áreas de memoria que pueden distinguirse:

- Código estático. Incluye todo el código de la tarea presente en su archivo ejecutable. No incluye por tanto el código de las bibliotecas con las que enlaza en tiempo de ejecución, denominadas bibliotecas de enlace dinámico. En el caso de la figura 2, se corresponde con el rango de memoria asociado al archivo `3-4maps-1` con atributos `r-xp`.
- Datos estáticos. Se corresponden con las constantes y variables globales de la tarea, también definidas en el archivo ejecutable. En el caso de la figura 2 se corresponde con los rangos de memoria asociados al archivo `3-4maps-1` con atributos `r--p` para constantes y `rw-p` para variables.
- Pila (*stack*). Entre otras cosas, almacena las variables locales de la tarea así como las direcciones de retorno de las funciones. La ubicación de la pila en los sistemas operativos Linux no la fijan las herramientas de desarrollo. La ubicación es aleatoria y la fija el sistema operativo durante la carga de la tarea para dificultar los ataques por desbordamiento de la pila. (a esta técnica de seguridad se la conoce como *Address Space Layout Randomization* (ASLR) y se puede encontrar más información en ASLR). En el caso de la figura 2 se corresponde con el rango de memoria marcado como `[stack]`.
- Montón o montículo (*heap*). Es un área reservada para memoria solicitada dinámicamente. Para reservar memoria en el montón se emplea típicamente la función `malloc()` de C. En el caso del programa `3-4maps-1` no existe montón.
- Bibliotecas de enlace dinámico. El ejecutable de la tarea contiene código que hace de interfaz con las bibliotecas dinámicas. El enlace con estas bibliotecas se hace en tiempo de carga (cuando se lanza el programa) o en tiempo de ejecución. En este último caso se habla de carga dinámica, lo cual permite generar el ejecutable sin que siquiera existan las bibliotecas, descubrirlas en tiempo de ejecución, cargarlas y finalmente descargarlas cuando no hacen falta.

  Las bibliotecas dinámicas en Linux tienen típicamente la extensión `.so`. Por ejemplo, en la figura 2 se observa la biblioteca `libc-2.23.so`, la biblioteca de C estándar en su versión dinámica. También se puede observar la biblioteca `ld-2.23.so`. Se trata de la biblioteca dinámica necesaria para llevar a cabo el enlace dinámico con las demás bibliotecas dinámicas. ¿Qué tamaño total en bytes ocupa la biblioteca estándar de C en su versión dinámica? Responde en el cuestionario: pregunta 1.
- VDSO (*Virtual Dynamic Shared Object*). Se trata de una pequeña zona de memoria que contiene una librería dinámica, la cual permite implementar de forma muy eficiente llamadas a servicios que no requieren privilegios para ser ejecutados, evitando la sobrecarga de los cambios de contexto entre la tarea y Linux. Por ejemplo, servicios para obtener la hora o las características de la CPU.
- VVAR (*Virtual Dynamic Shared Object*). Zona de memoria de sólo lectura, compartida con el sistema operativo, que contiene datos a los que acceden las rutinas almacenadas en el área VDSO.

El programa `3-4maps-1` utiliza la biblioteca estándar de C en su versión dinámica. La razón es que el compilador `gcc` enlaza por defecto con esta biblioteca. Si se desea enlazar con la versión estática de la librería (`libc.a`) debe indicarse de forma explícita añadiendo la opción `-static`.

- Compila y enlaza el programa `3-4maps.c` usando bibliotecas estáticas, empleando la siguiente orden

```
$> gcc -static 3-4maps.c -o 3-4maps-2
```

- Ejecuta el archivo resultante, visualiza su archivo *maps* y observa los cambios experimentados con respecto a la versión estática, usando la figura 2 como referencia.
- Compara los tamaños de los archivos ejecutables `3-4maps-1` y `3-4maps-2` usando la siguiente orden.

```
$> ls -l 3-4maps-*
```

  ¿Qué diferencia de tamaño hay en KiB? Responde en el cuestionario: pregunta 2.

  Esta diferencia es debida a que al código del programa `3-4maps-2` se ha añadido código de la biblioteca estándar porque se ha enlazado estáticamente.

Si en lugar de conocer la organización del espacio de direcciones virtuales de una tarea se desease conocer la organización del espacio de direcciones físicas del sistema, se podría volcar el archivo del sistema `/proc/iomem`. En dicho archivo se pueden encontrar por ejemplo las interfaces de periféricos ubicadas en memoria.

2. Análisis del espacio de direcciones de una tarea

Vamos a analizar las direcciones virtuales y físicas de los siguientes elementos de una tarea:

- Variables globales.
- Variables locales.
- Funciones.

La tabla de páginas de cualquier tarea se encuentra en el área de memoria asociada al sistema operativo, es decir, en el rango de direcciones virtuales `C0000000h–FFFFFFFFh` y por lo tanto es inaccesible para las tareas. Para obtener las direcciones físicas emplearemos la función `get_pte()`, la cual se comunica con un módulo del núcleo (*driver*) denominado `linmem` que le permite acceder a la tabla de páginas de la tarea y por lo tanto obtener las direcciones físicas. El módulo `linmem` puede acceder a dichas direcciones, pues al formar parte del sistema operativo se ejecuta con su nivel de privilegio. Básicamente, el módulo le devuelve a la tarea la entrada en la tabla de páginas (ETP) asociada a una dirección virtual.

Una tarea puede comunicarse con dicho módulo de forma muy sencilla empleando la biblioteca `libmem.a`, que exporta la función `get_pte()`. Para usar dicha función, basta con incluir el archivo de cabecera `atc/linmem.h` en el código fuente y emplear la opción `-lmem` dentro de la orden de compilación y enlazado.

La función `get_pte()` proporciona para una dirección virtual dada el valor de la ETP asociada. La función tiene el siguiente prototipo o firma:

```
int get_pte(void *virtual_addr, unsigned int *p_pte);
```

- `virtual_addr` es la dirección virtual para la cual se quiere extraer la información.
- `p_pte` es la dirección en la que la función escribe el valor de la ETP de la dirección virtual. Si se pasa como dirección el valor `NULL`, la función no escribe nada en este parámetro y sirve para comprobar si el módulo `linmem` está operativo.

La función devuelve 0 cuando se ejecuta sin errores y devuelve -1 cuando se produce un error. Se puede incluir el archivo `errno.h` y consultar la variable `errno` en él definida para conocer la causa del error.

A partir de la función `get_pte()` vamos a construir una función que imprime los datos asociados a una determinada dirección de memoria virtual.

- El módulo `linmem` está disponible en la máquina virtual suministrada para realizar las prácticas. No obstante, puede no estar activo si se ha actualizado el núcleo o *kernel* de Linux. Comprueba si está activo con esta orden:

```
$> systemctl status linmem
```

> Si el estado no es activo, hay que recompilar el módulo y volver a instalarlo con estas órdenes
>
> ```
> $> cd /usr/src/atc/linmem
> $> sudo make clean
> $> sudo make install
> ```
>
> Comprueba con la orden explicada anteriormente que ahora está activo.

- Edita el fichero `3-4print-vm.c` para ver su contenido. Este fichero simplemente utiliza la función `print_virtual_physical_pte()` para imprimir la información de una dirección virtual. El prototipo de esta función está en `3-4print-pte.h` y su implementación en `3-4print-pte.c`.
- Edita el fichero `3-4print-pte.c` para ver su contenido. Debes observar cómo hemos incluido el fichero de cabecera `atc/linmem.h`. Recuerda que es un fichero necesario para poder utilizar la función `get_pte()` que interactúa con `linmem` y que está implementada en la biblioteca estática `libmem.a` de la máquina virtual de la asignatura. El fichero incluye la función `print_virtual_physical_pte()`, la cual emplea la función `get_pte()` para poder escribir en pantalla una dirección virtual, la entrada en la tabla de páginas (ETP) asociada, la dirección física asociada y los *flags* de propiedades asociados a esa página.
- Para que la función funcione correctamente debes completar los huecos <<**Complete 1**>> y <<**Complete 2**>>. En estos huecos se calculan los *flags* asociados a la página de memoria y la dirección física asociada a la dirección virtual que se le pasa como parámetro a la función. Toda esa información se obtiene a partir de la entrada de la tabla de páginas devuelta por `get_pte()`.

  Como se muestra en la figura 3, el marco de página asociado a la página virtual se encuentra en los 20 bits más significativos de la ETP, mientras que los 12 bits restantes constituyen los *flags* que definen las características de la página. En el apartado La tabla de páginas en GNU/Linux se entrará en detalle en la función de los diferentes bits que componen los *flags*.

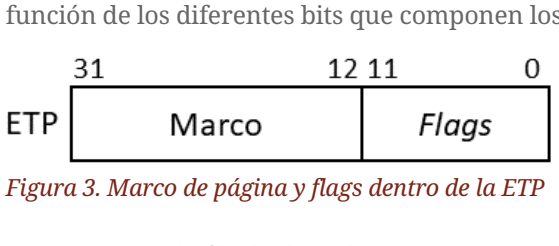


Figura 3. Marco de página y flags dentro de la ETP

  Para realizar el cálculo de cada uno de esos parámetros tendremos que usar el operador `&` del lenguaje C que realiza la operación `and` bit a bit entre dos operandos y realizar algún *casting*
- Compila conjuntamente los ficheros `3-4print-vm.c` y `3-4print-pte.c` con la siguiente orden:

```
$> gcc 3-4print-vm.c 3-4print-pte.c -o 3-4print-vm -lmem
```

- Ejecuta el programa para comprobar su salida y comprueba que los valores son coherentes.

Vamos a utilizar la función `print_virtual_physical_pte()` creada para analizar algunas de las zonas del espacio de direcciones de una tarea.

- Edita el fichero `3-4proclinux-1.c` para ver su contenido. Como se puede observar, el programa define una variable local (que se ubica en la sección de pila de la tarea) y una variable global (que se ubica en la sección de datos de la tarea). Además, dicho programa escribe por pantalla las direcciones virtuales, físicas y *flags* asociados a la variable local, la variable global y la propia función `print_virtual_physical_pte()`.
- Abre una terminal y desde la misma compila y enlaza el programa `3-4proclinux-1.c` con la orden.

```
$> gcc -Wall 3-4proclinux-1.c 3-4print-pte.c -o 3-4proclinux-1 -lmem
```

  La opción `-Wall` hace que se muestren todos los avisos de compilación. Si todo va bien, se genera un archivo ejecutable de nombre `3-4proclinux-1`.
- A continuación debes ejecutar simultáneamente el fichero ejecutable generado desde dos terminales distintas y responder a las siguientes preguntas: ¿Coinciden las direcciones virtuales de las variables locales de ambos programas? ¿Por qué? Responde en el cuestionario: pregunta 3.

  ¿Coinciden las direcciones virtuales de las variables globales de ambos programas? ¿Por qué? Responde en el cuestionario: pregunta 4.

  En ambos casos, ¿coinciden las direcciones físicas? Responde en el cuestionario: pregunta 5.
- Lleva a cabo la misma observación sobre las direcciones virtuales de la función `print_virtual_physical_pte()` de las dos instancias. Observarás que de nuevo coinciden pero, ¿coinciden las direcciones físicas? ¿Cómo explicas este comportamiento? Responde en el cuestionario: pregunta 6.

3. Asignación dinámica de memoria

Cuando una tarea arranca, el sistema operativo le asigna la memoria necesaria para ubicar sus datos y código. No obstante, en muchos casos durante la realización de un programa no se conocen sus necesidades futuras de memoria, pues éstas pueden depender por ejemplo de los datos a procesar. Aunque podrían definirse dentro del programa estructuras de datos de tamaño suficiente considerando el caso más desfavorable, esta política supone un desaprovechamiento excesivo de memoria y una mala práctica de programación.

Para resolver el problema anterior se proporcionan servicios que permiten que una tarea, a lo largo de su ejecución, pueda pedir más memoria cuando la necesita y liberarla más adelante cuando ya no le hace falta.

Si la memoria se solicita al sistema operativo, éste puede establecer restricciones sobre la máxima cantidad de memoria que puede asignar a una tarea, así como la proporción de reparto de ésta entre la memoria física y el archivo de paginación. La memoria solicitada por una tarea al sistema operativo se asocia a un rango de direcciones virtuales que puede ser especificado por la tarea en el momento de llamar al servicio, aunque lo habitual es dejar libertad al sistema operativo para que elija el rango de direcciones virtuales de la tarea más apropiado.

Vamos a ver dos formas de llevar a cabo la petición de memoria. La primera, más compleja pero también más potente, llama a un servicio específico de la API (*Application Programming Interface*) del sistema operativo y se corresponde con la descripción del párrafo anterior. La segunda llama a una función de usuario de C y por lo tanto incurre en una sobrecarga menor.

3.1. El servicio mmap de POSIX

Se trata de un servicio POSIX que permite solicitar memoria dentro del sistema operativo Linux. Asimismo, permite mapear archivos en memoria, pero esta funcionalidad no la emplearemos en esta práctica, por lo que la ignoraremos.

> POSIX engloba un conjunto de estándares de interfaz con el sistema operativo. Básicamente persiguen el objetivo de que una aplicación pueda compilarse y ejecutarse directamente sobre diferentes sistemas operativos.

En primer lugar, antes de llamar a la función `mmap()` debe crearse un descriptor de memoria y configurarlo con el tamaño de la memoria a solicitar. El siguiente fragmento de código muestra cómo solicitar un área de memoria de 16 KiB.

```
char string[30]; /* String with the name of the memory chunk */
void *p;         /* Pointer to the requested memory chunk */
int memd;        /* Requested memory descriptor */

/* Open a memory descriptor based on the PID */
sprintf(string, "/mem-%d", getpid());
if ((memd = shm_open(string, O_RDWR|O_CREAT|O_EXCL, S_IRWXU)) < 0)
    return -1; /* Exit if error */

/* Define the size of the memory chunk */
if (ftruncate(memd, 16*1024) < 0)
{
    shm_unlink(string); /* Close the memory descriptor */
    return -1; /* Exit if error */
}

/* Map the memory chunk in the virtual memory area of the task */
if ((p = mmap(NULL, 16*1024, PROT_READ|PROT_WRITE, MAP_SHARED, memd, 0)) == MAP_FAILED)
{
    shm_unlink(string); /* Close the memory descriptor */
    return -1; /* Exit if error */
}
```

La función `shm_open()` crea un descriptor de memoria con un cierto nombre y unos parámetros de acceso al mismo. El nombre debe comenzar con el carácter *"/"*.

La función `ftruncate()` configura el descriptor de memoria con un tamaño de 16 KiB. Una vez ejecutada esta función tenemos un descriptor de memoria configurado, pero la tarea aún no ha recibido los 16 KiB de memoria.

A continuación vamos a describir los parámetros empleados en la llamada a la función `mmap()`, así como el valor retornado.

- `NULL`. Indica que el sistema operativo elige la dirección virtual de comienzo de la memoria que va a proporcionar a la tarea. Si especificásemos una dirección, estaríamos pidiendo al sistema operativo que mapease la memoria a partir de esa dirección virtual.
- `16*1024`. Es el tamaño del área de memoria a mapear. No tiene por qué coincidir con el valor empleado en la llamada a `ftruncate()`, pues podríamos por ejemplo pedir primero 4 KiB y después pedir los 12 KiB restantes.
- `PROT_READ|PROT_WRITE`. Indica que el área de memoria que le asignará el sistema operativo será de lectura y escritura. Por ejemplo, en este caso no se podría ejecutar código en dicho área de memoria.
- `MAP_SHARED`. Indica que la memoria puede ser accedida desde otras tareas. Por lo tanto, esta opción permite la compartición del área de memoria solicitada por aquellas tareas que crearán sus descriptores de memoria usando el mismo nombre y eliminando la opción `O_CREAT`. Si se desea impedir la posibilidad de compartición con otra u otras tareas debería especificarse la opción `MAP_PRIVATE`.
- `memd`. Es el descriptor de memoria, previamente creado y configurado.
- `0`. Especifica el desplazamiento dentro del elemento a mapear. En nuestro caso siempre será cero.
- En caso de éxito, la función devuelve la dirección virtual a partir de la cual el sistema operativo ha mapeado la memoria solicitada. En caso contrario, la función devuelve el valor `MAP_FAILED`.

Puede encontrarse información más detallada sobre la función `mmap()` consultando los manuales de Linux.

- Edita el fichero `3-4proclinux-2.c` para ver su contenido. Como puedes observar, el programa solicita 16 KiB de memoria al sistema operativo durante su ejecución y entre otras cosas escribe por pantalla la dirección virtual y física asociada a la primera dirección del área de memoria solicitada.
- Compila el programa anterior con la orden:

```
$> gcc -Wall 3-4proclinux-2.c 3-4print-pte.c -o 3-4proclinux-2 -lrt -lmem
```

  Observa la opción `-lrt`, que especifica el enlazado con la biblioteca de tiempo real, en la cual están definidas las funciones `shm_open()` y `shm_unlink()`.
- Ejecuta el programa obtenido y observa desde una segunda terminal su archivo `maps`. Localiza la zona de memoria reservada y comprueba que el tamaño de la misma y los *flags* se corresponden con los parámetros pasados a la función `mmap()`.

  ¿Qué ocurre con la dirección física de comienzo del área de memoria solicitada después de la petición de memoria? Responde en el cuestionario: pregunta 7.

  La razón de este comportamiento es que todavía no se ha accedido a la página y, por lo tanto, el sistema no le ha asignado direcciones físicas. Vamos a comprobar qué ocurre tras acceder a la variable.
- Copia el programa `3-4proclinux-2.c` a `3-4proclinux-3.c` y edita este último. Justo después de la llamada a la función `mmap()`, si el retorno ha sido correcto, debes añadir el código necesario para que se escriba un carácter en la primera posición de memoria del área concedida por el sistema operativo. Para hacerlo puedes emplear por ejemplo la instrucción

```
*(char *)p = 'A';
```

- Compila el programa `3-4proclinux-3.c` (no te olvides de incluir `3-4print-pte.c`) y ejecútalo. ¿Qué ocurre ahora con la dirección física después de la petición de memoria y la escritura del carácter? ¿Cómo lo explicas? Responde en el cuestionario: pregunta 8.

3.2. El servicio munmap de POSIX

Se trata de un servicio del estándar POSIX que solicita al sistema operativo la liberación de memoria previamente concedida.

- Copia el programa `3-4proclinux-3.c` a `3-4proclinux-4.c` y edita este último. Justo después de la instrucción que escribe en la primera página solicitada debes añadir la instrucción

```
munmap(p, 16*1024);
```

  que libera la memoria previamente solicitada. El primer parámetro identifica la dirección virtual de comienzo del área de memoria a liberar. El segundo parámetro identifica el tamaño del área a liberar. Debe tenerse en cuenta que no es necesario liberar todo el área solicitada de una sola vez.
- Compila el programa `3-4proclinux-4.c` y ejecútalo. ¿Qué ocurre ahora con la dirección física después de la liberación de memoria? ¿Por qué? Responde en el cuestionario: pregunta 9. Observa además desde la segunda terminal cómo ha desaparecido la zona de memoria solicitada previamente.

  ¿Qué crees que ocurriría si justo después de ejecutar `munmap()` volviésemos a ejecutar la instrucción `*(char *)p = 'A'`? Responde en el cuestionario: pregunta 10

3.3. Las funciones malloc() y free()

Dentro del lenguaje C se definen dos funciones estándar denominadas `malloc()` y `free()` que, aunque mucho menos potentes que las funciones POSIX `mmap()` y `mummap()`, permiten llevar a cabo mucho más fácilmente la solicitud y liberación dinámica de memoria. Además, al formar parte del estándar C hacen que el código sea portable a multitud de plataformas.

La implementación de la función `malloc()` depende del sistema operativo, aunque lo habitual es que emplee el montón (*heap*) para ubicar la memoria solicitada. En este caso la asignación de memoria es muy rápida pues no involucra la llamada a ningún servicio del sistema operativo.

> Debes recordar que la llamada a un servicio del sistema operativo requiere operaciones de preparación y finalización, como guardar el estado de la tarea en ejecución y recuperarlo posteriormente. No obstante, en la práctica la penalización puede ser poco relevante en comparación al tiempo de ejecución del programa.

La función `malloc()` recibe como parámetro el número de bytes del área de memoria a solicitar y devuelve la dirección virtual de comienzo a partir de la cual se ubica dicho área de memoria. Cuando ocurre un error durante su ejecución devuelve `NULL`.

En los sistemas operativos Linux es habitual que por debajo de un cierto umbral de tamaño de memoria solicitada la asignación de memoria proceda del montón. Por encima de dicho umbral la función `malloc()` asigna la memoria llamando internamente a la función `mmap()`.

Una vez que el área de memoria solicitada ya no es necesaria se debe liberar, para lo cual se emplea la función `free()`. Esta función debe recibir como parámetro la dirección devuelta previamente por la función `malloc()`.

- Edita el archivo `3-4proclinux-5.c` para ver su contenido. Se trata de un programa que solicita 16 KiB de memoria al sistema operativo empleando la función `malloc()` y muestra las direcciones virtual y física de comienzo de dicho área.
- Compila el programa anterior y ejecútalo sobre una terminal. Verás que está esperando por la pulsación de la tecla `ENTER`. Esto nos permitirá consultar su archivo `maps`.

  Observa cómo la memoria física ha sido asignada sin necesidad de haber accedido previamente a la misma, al contrario de lo que ocurría con la función `mmap()`.
- Observa desde otra terminal el contenido del archivo *maps* de la tarea anterior. ¿En qué zona de memoria de la tarea se encuentra ubicado el área de memoria solicitada? Responde en el cuestionario: pregunta 11.

  Observa además cómo la ubicación dentro del espacio de direcciones virtuales no tiene posibilidad de elegirla la tarea, se selecciona automáticamente. Esta ubicación es devuelta por la función `malloc()`.
- Edita de nuevo el archivo `3-4proclinux-5.c`, pero ahora guárdalo con el nombre `3-4proclinux-6.c`. Debes modificarlo para solicitar ahora 256 KiB de memoria. Compílalo, ejecútalo y observa el contenido de su archivo *maps*. ¿Que ha ocurrido? ¿En qué zona se ha asignado la memoria solicitada? ¿Por qué? Responde en el cuestionario: pregunta 12.

4. La tabla de páginas en GNU/Linux

La gestión de memoria en la arquitectura x86-32 es bastante compleja en comparación a otras arquitecturas. Admite diferentes esquemas de paginación e incluso diferentes tamaños de página dentro del mismo esquema. La máquina empleada en prácticas emplea un esquema de paginación con dos tamaños de página posibles: 4 KiB, que requiere traducción en dos pasos, y 4 MiB, que se puede traducir en un paso.

Cada tamaño de página tiene ventajas y desventajas. Por un lado, con el uso de páginas de 4 KiB el desperdicio de memoria es menor. Por ejemplo, una tarea que requiere 3 KiB desperdicia 1 KiB con páginas de tamaño 4 KiB y casi 4 MiB con páginas de tamaño 4 MiB. Sin embargo, el núcleo de un sistema operativo como Linux requiere gran cantidad de memoria, por lo que el empleo sólo de páginas de tamaño 4 KiB exigiría un gran número de páginas y un gran número de entradas en el TLB, reduciendo el número de entradas en el TLB para las tareas. La consecuencia final sería que la tasa de aciertos del TLB se reduciría y aumentaría el tiempo medio de acceso a memoria.

Por simplicidad nos centraremos en las páginas de 4 KiB e ignoraremos las de 4 MiB. La figura 4 forma parte de la documentación oficial de Intel e indica la organización de los bits de una entrada en tabla de páginas (ETP) para páginas de 4 KiB.

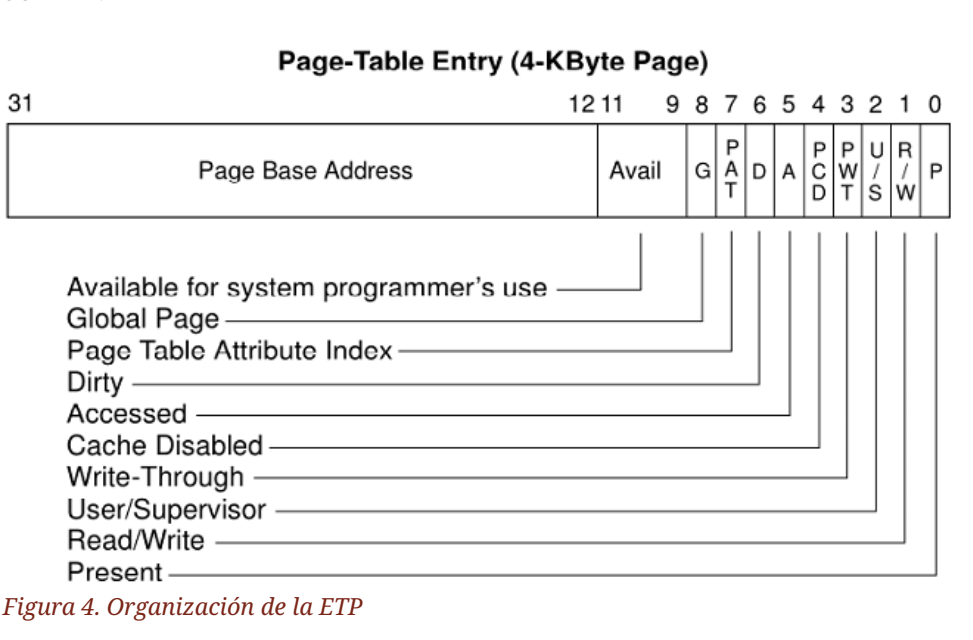


Figura 4. Organización de la ETP

Los bits 31 a 12 (`Page Base Address`) de la ETP contienen el marco de página y por lo tanto la traducción de la página virtual cuando el bit de presencia *P* está a 1. El resto de bits constituyen los *flags*, empleados para establecer las características de la página. Los *flags* más destacados son estos:

- Bit 0 (`P`). Se trata del bit de presencia. Cuando está a 1 la página está en memoria y los bits 31 a 12 contienen un marco de página válido.
- Bit 1 (`R/W`). Cuando toma el valor 0 la página es de sólo lectura, mientras que cuando toma el valor 1 es de lectura y escritura.
- Bit 2 (`U/S`). Cuando toma el valor 0 la página es de supervisor, esto es, sólo puede acceder a la misma el sistema operativo. Cuando toma el valor 1 la tarea también puede acceder a ella.
- Bit 3 (`PWT`). Cuando está a 1 y está habilitada la caché indica que la página emplea una estrategia de escritura *write-through* para la página. Cuando está a cero emplea la estrategia *write-back*.
- Bit 4 (`PCD`). Cuando está a 1 la caché está deshabilitada para la página y por lo tanto cualquier acceso va directamente a memoria principal. En caso contrario, la caché está habilitada.
- Bit 5 (`A`). Se pone a 1 cuando la página es accedida, ya sea en lectura o en escritura. Una vez se pone a 1 no se pone a 0 hasta que el sistema operativo reinicia este bit escribiendo sobre la ETP.
- Bit 6 (`D`). Es análogo al bit de *dirty* de las cachés. Cuando se escribe en la página este bit se pone a 1, de tal forma que si en un futuro es necesario reemplazar la página en memoria para llevar una nueva página, la página reemplazada no se descarta, sino que se guarda en disco.
- Bit 8 (`G`). Este bit cuando está a 1 marca la ETP como global. Por lo tanto, la ETP es válida para todas las tareas y no es necesario vaciar la copia de esta ETP en el TLB cada vez que se produce un cambio de contexto de una tarea a otra. Las páginas del sistema operativo suelen tener este bit a 1.

Con esta información vamos a analizar el contenido de las entradas en la tabla de páginas para distintas zonas de la memoria virtual.

- Ejecuta de nuevo el programa `3-4proclinux-1`. ¿Qué valor tienen los *flags* de la página virtual de la variable global? Responde en el cuestionario: pregunta 13.

  ¿Por qué crees que coinciden con los *flags* de la página asignada a la variable local? Responde en el cuestionario: pregunta 14.

  ¿Qué valor tienen los *flags* de la página virtual asignada a la función? Responde en el cuestionario: pregunta 15.

  ¿Por qué crees que no coinciden con los *flags* de las variables de memoria? Responde en el cuestionario: pregunta 16.
- Vamos a comprobar si las suposiciones realizadas son correctas. En una hoja de papel copia el esquema del contenido de una entrada en la tabla de páginas que se muestra en la figura 4. Ten la precaución de hacer más amplias las casillas de los bits `0` a `7` (para poder escribir en su interior) y coloca en la parte superior el número de bit y en la inferior el contenido de la casilla que se ve en la figura 4.

  Coloca los bits de los *flags* de la variable global en las casillas que les corresponda. ¿Qué valor tienen los *flags* de la página asignada a la variable global? Responde en el cuestionario: pregunta 17.

  Al coincidir los *flags* de la variable global con los de la variable local, el mismo esquema te sirve para ésta última.
- Crea un nuevo esquema para los *flags* de la función. ¿Qué valores tienen los distintos *flags*? Responde en el cuestionario: pregunta 18.

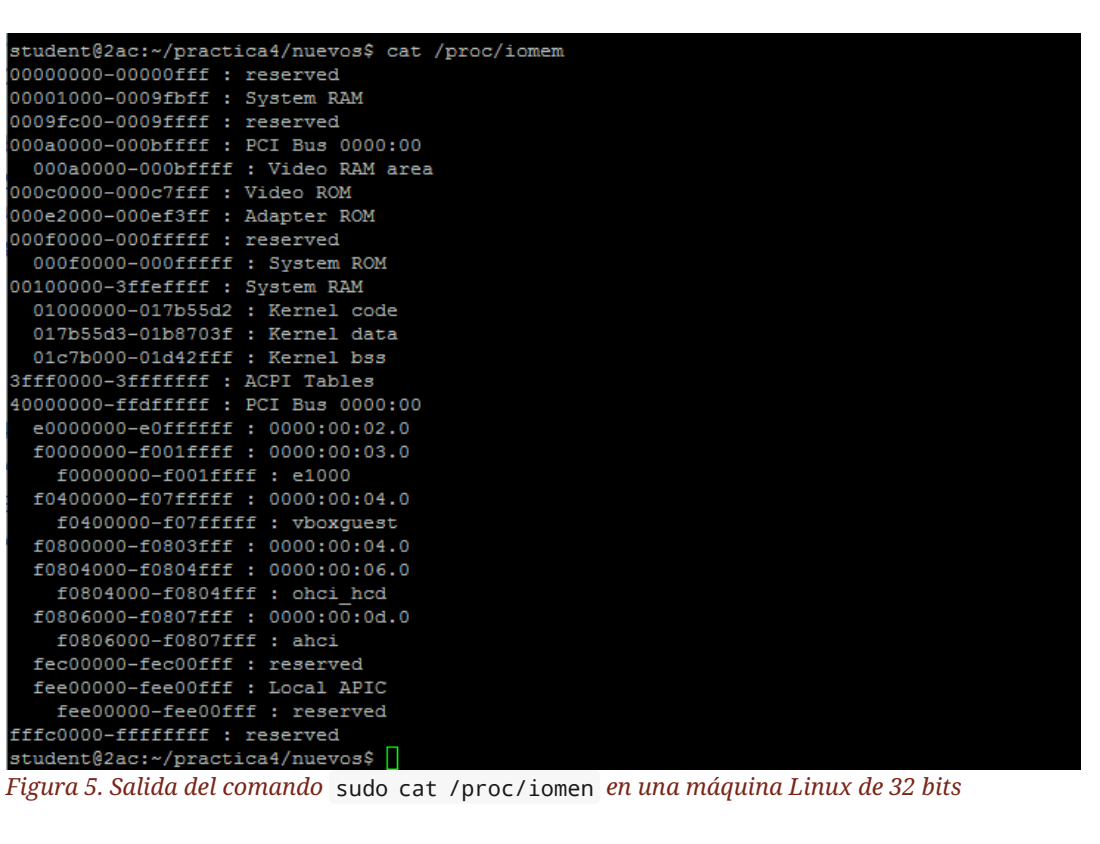
  ¿Qué *flags* son diferentes respecto a las páginas de las variables? ¿Por qué? Responde en el cuestionario: pregunta 19.

Vamos ahora a ver las propiedades de las zonas de memoria asignadas al sistema operativo, centrándonos únicamente en sus páginas de 4 KBi.

- Ejecuta el comando

```
$> sudo cat /proc/iomem
```

El archivo *iomem* contiene las direcciones físicas asignadas a diferentes elementos del sistema. El archivo tiene un aspecto similar al de la figura 5.



```
student@2ac:~/practica4/nuevos$ cat /proc/iomem
00000000-00000fff : reserved
00001000-0009fbff : System RAM
0009fc00-0009ffff : reserved
000a0000-000bffff : PCI Bus 0000:00
  000a0000-000bffff : Video RAM area
000c0000-000c7fff : Video ROM
000e2000-000ef3ff : Adapter ROM
000f0000-000fffff : reserved
  000f0000-000fffff : System ROM
00100000-3ffeffff : System RAM
  01000000-017b55d2 : Kernel code
  017b55d3-01b8703f : Kernel data
  01c7b000-01d42fff : Kernel bss
3fff0000-3fffffff : ACPI Tables
40000000-ffdfffff : PCI Bus 0000:00
  e0000000-e0ffffff : 0000:00:02.0
  f0000000-f001ffff : 0000:00:03.0
    f0000000-f001ffff : e1000
  f0400000-f07fffff : 0000:00:04.0
    f0400000-f07fffff : vboxguest
  f0800000-f0803fff : 0000:00:04.0
  f0804000-f0804fff : 0000:00:06.0
    f0804000-f0804fff : ohci_hcd
  f0806000-f0807fff : 0000:00:0d.0
    f0806000-f0807fff : ahci
  fec00000-fec00fff : reserved
  fee00000-fee00fff : Local APIC
    fee00000-fee00fff : reserved
fffc0000-ffffffff : reserved
student@2ac:~/practica4/nuevos$
```

Figura 5. Salida del comando `sudo cat /proc/iomen` *en una máquina Linux de 32 bits*

En el contenido del archivo *iomem* podemos ver que hay tres zonas en memoria física asociadas a elementos del núcleo de Linux: *kernel code*, *kernel data* y *kernel bss*. En esas zonas de la memoria física es dónde el sistema operativo aloja su código, sus datos inicializados y sus datos sin inicializar. Los datos inicializados se corresponden con variables inicializadas dentro del código fuente del núcleo de Linux, mientras que los datos no inicializados se corresponden con variables no inicializadas y otras estructuras de memoria que no requieren inicialización, como puede ser la pila. Un ejemplo de variable inicializada en un programa es `int a = 5;` mientras que una variable no inicializada sería `int b ;`.

Como hemos visto en la figura 1, de los 4 GiB de memoria virtual asignados a una tarea, el sistema operativo se mapea en el GiB de memoria virtual superior, a partir de la dirección de memoria `0xC0000000`. Para facilitar las cosas, el gestor de memoria Linux programa las ETP asociadas a las páginas del núcleo para que sean las mismas que las direcciones físicas en las que se encuentran, sumándoles el valor `0xC0000000`. Por ejemplo, si el código del núcleo está almacenado a partir de la dirección física `0x01000000`, esto significa que está mapeado a partir de la dirección virtual `0xC1000000`. en el espacio de direcciones virtuales de todas las tareas.

- Edita el programa `3-4kernelpages.c` y trata de entender su funcionamiento. Se trata de un programa que muestra el marco de página y los *flags* asociados a la primera página de 4 KiB que se encuentra en memoria física para las zonas de código, datos inicializados y datos sin inicializar del núcleo de Linux. El programa dispone de dos funciones aparte de la función principal: `get_kernel_phy_addr()` y `get_kernel_pages_memory()`. La primera accede al archivo `/proc/iomem` para obtener los rangos de direcciones físicas de las distintas áreas del núcleo de Linux, mientras que la última usa estas áreas para obtener para cada una la primera página virtual en memoria física.
- Al final del programa hay un hueco con el texto <<**Complete**>> que debes rellenar para imprimir por pantalla el marco de página y los *flags* asociados a cada una de las áreas de memoria física obtenidas por la función `get_kernel_phy_addr()`.
- Compila el programa y ejecútalo.
- Fíjate en la página virtual y en el marco de página de la primera página de código del núcleo de linux que se encuentra memoria física. ¿Qué rango de direcciones virtuales y físicas ocupan? ¿Ves alguna relación entre los dos tipos de direcciones? Responde en el cuestionario: pregunta 20
- Crea un esquema para los *flags* de esa página de código y analízalos. ¿Qué valores tienen los distintos *flags*? Responde en el cuestionario: pregunta 21. El valor del *flag dirty* en las páginas del núcleo de Linux es irrelevante y debe ignorarse, pues las páginas del núcleo de Linux permanecen en memoria y no se llevan a disco.
- Fíjate en la página virtual y en el marco de página de la primera página de datos inicializados del núcleo de linux que se encuentra en memoria física. ¿Qué rango de direcciones virtuales y físicas ocupan? ¿Ves de nuevo alguna relación entre los dos tipos de direcciones?

  Es posible que te salgan los mismos *flags* que en la página de código. Si ese es tu caso, ¿cuál crees que es la razón? Responde en el cuestionario: pregunta 22.

  El resto de páginas del área datos inicializados, ¿tendrán todos los mismos *flags* que los de la primera página? Responde en el Responde en el cuestionario: pregunta 23. Puedes comprobar tu respuesta modificando el programa para que muestre los *flags* de la última página de datos inicializados en lugar de la primera.
- Fíjate en la página virtual y en el marco de página de la primera página de datos no inicializados del núcleo Linux en memoria física. ¿Qué rango de direcciones virtuales y físicas ocupan? ¿Ves alguna relación entre los dos tipos de direcciones?
- Crea un esquema para los *flags* de esa página de datos no inicializados y analízalos. ¿Qué valores tienen los distintos *flags*? Responde en el cuestionario: pregunta 24.

  ¿El *flag* de escritura es el esperado? ¿Por qué? Responde en el cuestionario: pregunta 25.

Fíjate que en todas las páginas de memoria asociadas al núcleo el bit *Global* estaba activo. Por lo tanto, las entradas del TLB correspondientes son válidas para todas las tareas.

Archivos de la práctica

- Añade al repositorio los archivos que no los estén con `git add <archivo>` (recuerda que no es necesario subir archivos ejecutables, solo archivos en código fuente), confirma los cambios realizados con `git commit` y súbelos al repositorio remoto en Bitbucket con `git push`.

Ejercicios

- Tomando como referencia el programa `3-4proclinux-3.c`, crea un nuevo programa de nombre `3-4proclinux-7.c` que comparta el área de memoria creada por `3-4proclinux-3.c` y muestre como carácter el contenido de la primera posición de memoria del área solicitada. Recuerda que debes emplear exactamente la misma cadena para la creación del descriptor de memoria y eliminar la opción `O_CREAT`, pues el área de memoria compartida ya ha sido creada. Para conocer el nombre exacto de la cadena puedes volcar el contenido del archivo *maps* correspondiente a `3-4proclinux-3`. Puedes tomar como referencia el siguiente fragmento de código y sustituir el número `1234` por el PID de `3-4proclinux-3`.

```
/* Open a memory descriptor */
if ((memd = shm_open("/mem-1234", O_RDWR|O_EXCL, S_IRWXU)) < 0)
    return -1; /* Finish the program in case of error */
```

  Además, debes sustituir las líneas que escribían un carácter en el área compartida por las líneas que lo leen de dicho área. Por ejemplo, puedes sustituirlas por las siguientes líneas:

```
/* Read a character from the first location in the allocated memory */
printf("The character written by the other task is: %c\n", *((char *)p));
```

- Compila y ejecuta el programa `3-4proclinux-7` sin terminar el `3-4proclinux-3`, pues pretendemos que compartan memoria. ¿Coincide el carácter mostrado con el que habías escrito con el programa `3-4proclinux-3.c`?
- ¿Qué ocurre con las direcciones físicas del área de memoria compartida mostradas por ambos programas?
- ¿Las direcciones virtuales del área de memoria compartida son las mismas para los dos programas? ¿Esto supone algún problema?

---

[1] Este módulo, desarrollado por los profesores de la asignatura, no forma parte de la distribución de Linux.